Material de Estudo
Estruturas de Dados II

Prof. Tiago Pessoa Ferreira de Lima

Conteúdo

1 Revisão Rápida: Árvores Binárias **2**

2 Questão 1: Média por Nível **2**

3 Questão 2: Caminho de Soma Mínima **3**

4 Questão 3: Remover Folhas **5**

5 Questão 4: Verificar Espelho entre Duas Árvores **6**

6 Dicas Gerais para a Prova **8**

7 Exemplos de Testes **8**

1 Revisão Rápida: Árvores Binárias

É uma estrutura de dados hierárquica composta por nós. Cada nó possui:

- Um valor (dado)
- Uma referência para o filho esquerdo (`left`)
- Uma referência para o filho direito (`right`)

A **raiz** é o nó principal. Nós sem filhos são chamados de **folhas**. Percursos comuns:

- **Pré-ordem**: raiz → esquerda → direita
- **Em-ordem**: esquerda → raiz → direita
- **Pós-ordem**: esquerda → direita → raiz

A recursão é a ferramenta natural para trabalhar com árvores. Para percursos por nível, utiliza-se uma fila (busca em largura).

2 Questão 1: Média por Nível

Enunciado: Implemente o método `media_por_nivel(self)` na classe `ArvoreBinaria` que retorna uma lista com a média aritmética dos valores dos nós em cada nível da árvore. O primeiro elemento da lista corresponde à média do nível da raiz (nível 0), o segundo ao nível 1, e assim por diante. Se a árvore estiver vazia, retorne uma lista vazia.

Exemplo

Considere a árvore:

```
        10
       /  \
      5    20
     / \     \
    2   7     30
```

Níveis:

- Nível 0: [10] → média = 10
- Nível 1: [5, 20] → média = 12.5
- Nível 2: [2, 7, 30] → média = 13

Resultado: `[10, 12.5, 13]`

Estratégia

- Utilizar busca em largura (BFS) com uma fila.
- Processar um nível por vez: para cada nível, somar os valores e contar os nós.
- Calcular a média e adicionar à lista de resultados.
- Continuar até que todos os níveis sejam processados.

Implementação

```python
from collections import deque

class No:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None

class ArvoreBinaria:
    def __init__(self):
        self.raiz = None

    def media_por_nivel(self):
        if self.raiz is None:
            return []

        resultados = []
        fila = deque([self.raiz])

        while fila:
            qtd_nos = len(fila)
            soma = 0
            for _ in range(qtd_nos):
                no = fila.popleft()
                soma += no.valor
                if no.esquerda:
                    fila.append(no.esquerda)
                if no.direita:
                    fila.append(no.direita)
            media = soma / qtd_nos
            resultados.append(media)

        return resultados
```

Listing 1: Método $\text{media}_{p}or_{n}ivel$

Complexidade

- Tempo: $O(n)$ – cada nó é visitado exatamente uma vez.
- Espaço: $O(m)$ onde m é o número máximo de nós em um nível (largura da árvore).

3 Questão 2: Caminho de Soma Mínima

Enunciado: Implemente o método `caminho_soma_minima(self)` que retorna uma lista com os valores do caminho da raiz até uma folha cuja soma dos valores seja a **menor possível**. Se houver empate, retorne qualquer um dos caminhos com soma mínima. Caso a árvore esteja vazia, retorne uma lista vazia.

Exemplo

Considere a árvore:

```
        5
       / \
      4   8
     /   / \
    11  13  4
   /  \      \
  7    2      1
```

Caminhos raiz-folha:

- $5 \to 4 \to 11 \to 7$: soma $= 27$
- $5 \to 4 \to 11 \to 2$: soma $= 22$
- $5 \to 8 \to 13$: soma $= 26$
- $5 \to 8 \to 4 \to 1$: soma $= 18$

O caminho com soma mínima é `[5, 8, 4, 1]` (soma 18).

Estratégia

- Realizar uma busca em profundidade (DFS) recursiva.
- Manter o caminho atual e a soma atual.
- Ao chegar em uma folha, comparar a soma com a melhor (menor) encontrada até então.
- Guardar o melhor caminho.

Implementação

```python
def caminho_soma_minima(self):
    if self.raiz is None:
        return []

    melhor_soma = float('inf')
    melhor_caminho = []

    def dfs(no, caminho_atual, soma_atual):
        nonlocal melhor_soma, melhor_caminho
        if no is None:
            return

        caminho_atual.append(no.valor)
        soma_atual += no.valor

        if no.esquerda is None and no.direita is None:
```

```
            if soma_atual < melhor_soma:
                melhor_soma = soma_atual
                melhor_caminho = caminho_atual.copy()
        else:
            dfs(no.esquerda, caminho_atual, soma_atual)
            dfs(no.direita, caminho_atual, soma_atual)

        caminho_atual.pop()

    dfs(self.raiz, [], 0)
    return melhor_caminho
```

Listing 2: Método caminho$_soma_minima$

Complexidade

- Tempo: $O(n)$ no pior caso, pois percorremos todos os caminhos raiz-folha.
- Espaço: $O(h)$ para a pilha de recursão mais o caminho armazenado, onde h é a altura da árvore.

4 Questão 3: Remover Folhas

Enunciado: Implemente o método `remover_folhas(self)` na classe `ArvoreBinaria`. Este método deve percorrer a árvore em **pós-ordem** e remover **todas as folhas** (nós sem filhos), deixando os nós internos intactos. Após a remoção, a árvore deve continuar válida (nós que se tornaram folhas após remoção dos filhos não devem ser removidos nesta execução – a remoção é uma única passagem).

Exemplo

Árvore original:

```
        10
       /  \
      5    20
     / \     \
    2   7     30
```

Após `remover_folhas()`:

- Folhas originais: 2, 7, 30 são removidas.

Resultado:

```
        10
       /  \
      5    20
```

(Note que 5 e 20 agora são folhas, mas não são removidas porque a passagem já terminou.)

Estratégia

- Percorrer a árvore em pós-ordem (esquerda, direita, raiz).
- Para cada nó, verificar se ele é uma folha (ambos os filhos são `None`).
- Se for folha, retornar `None` para removê-lo (o pai deixará de referenciá-lo).
- Caso contrário, manter o nó, mas antes processar os filhos recursivamente.

Implementação

```python
def remover_folhas(self):
    def pos_ordem_remover_folhas(no):
        if no is None:
            return None
        # Primeiro processa os filhos
        no.esquerda = pos_ordem_remover_folhas(no.esquerda)
        no.direita = pos_ordem_remover_folhas(no.direita)
        # Se for folha, remove (retorna None)
        if no.esquerda is None and no.direita is None:
            return None
        # Caso contr rio, mant m o n
        return no

    self.raiz = pos_ordem_remover_folhas(self.raiz)
```

Listing 3: Método remover$_folhas$

Complexidade

- Tempo: $O(n)$ – cada nó é visitado uma vez.
- Espaço: $O(h)$ na pilha de recursão.

5 Questão 4: Verificar Espelho entre Duas Árvores

Enunciado: Implemente o método `sao_espelho(self, outra_arvore)` na classe `ArvoreBinaria` que recebe outra árvore binária e retorna `True` se as duas árvores são imagens espelho uma da outra, e `False` caso contrário. Duas árvores são espelho se:

- A raiz de uma é igual à raiz da outra.
- A subárvore esquerda da primeira é espelho da subárvore direita da segunda.
- A subárvore direita da primeira é espelho da subárvore esquerda da segunda.

Exemplo

Árvore A:

```
      1
     / \
    2   3
   /     \
  4       5
```

Árvore B (espelho de A):

```
      1
     / \
    3   2
   /     \
  5       4
```

`A.sao_espelho(B)` → `True`

Estratégia

- Criar uma função auxiliar recursiva que compara duas subárvores de forma espelhada.
- A comparação deve considerar:
  - Se ambas são `None`, são espelho.
  - Se apenas uma é `None`, não são espelho.
  - Se os valores são diferentes, não são espelho.
  - Caso contrário, verificar recursivamente: esquerda da primeira com direita da segunda, e direita da primeira com esquerda da segunda.
- O método principal chama essa função com as raízes das duas árvores.

Implementação

```python
def sao_espelho(self, outra_arvore):
    def espelho(no1, no2):
        if no1 is None and no2 is None:
            return True
        if no1 is None or no2 is None:
            return False
        if no1.valor != no2.valor:
            return False
        # Verifica espelhamento: esquerda de no1 com direita de no2
        # e direita de no1 com esquerda de no2
        return (espelho(no1.esquerda, no2.direita) and
                espelho(no1.direita, no2.esquerda))

    return espelho(self.raiz, outra_arvore.raiz)
```

Listing 4: Método sao$_e spelho$

Complexidade

- Tempo: $O(n)$ – cada nó das duas árvores é visitado uma vez.
- Espaço: $O(h)$ na pilha de recursão, onde h é a altura das árvores.

6 Dicas Gerais para a Prova

1. **Leia o enunciado com atenção** – entenda exatamente o que é pedido, inclusive casos especiais (árvore vazia, empates, etc.).
2. **Desenhe exemplos** – antes de codificar, teste mentalmente com árvores pequenas.
3. **Use recursão para percursos em profundidade** e filas para percursos em largura.
4. **Cuidado com mutabilidade** – ao passar listas como argumento, faça cópias quando necessário.
5. **Teste sua implementação** – após escrever o código, teste com os exemplos fornecidos e crie seus próprios casos de teste.
6. **Comente o código** – isso ajuda a organizar o raciocínio e mostra ao professor que você entendeu a lógica.

7 Exemplos de Testes

Você pode criar uma pequena classe de teste para verificar suas implementações:

```
# Quest o 1: M dia por n vel
arv1 = ArvoreBinaria()
arv1.raiz = No(10)
arv1.raiz.esquerda = No(5)
arv1.raiz.direita = No(20)
arv1.raiz.esquerda.esquerda = No(2)
arv1.raiz.esquerda.direita = No(7)
arv1.raiz.direita.direita = No(30)
print(arv1.media_por_nivel())  # [10, 12.5, 13]

# Quest o 2: Caminho soma m nima
arv2 = ArvoreBinaria()
# construir  rvore  do exemplo...
print(arv2.caminho_soma_minima())  # [5, 8, 4, 1]

# Quest o 3: Remover folhas
arv3 = ArvoreBinaria()
# popular...
arv3.remover_folhas()
# verificar resultado...

# Quest o 4: S o espelho?
arv4 = ArvoreBinaria()
```

```python
arv4.raiz = No(1)
arv4.raiz.esquerda = No(2)
arv4.raiz.direita = No(3)
arv4.raiz.esquerda.esquerda = No(4)
arv4.raiz.direita.direita = No(5)

arv5 = ArvoreBinaria()
arv5.raiz = No(1)
arv5.raiz.esquerda = No(3)
arv5.raiz.direita = No(2)
arv5.raiz.esquerda.esquerda = No(5)
arv5.raiz.direita.direita = No(4)

print(arv4.sao_espelho(arv5))  # True
```

Listing 5: Exemplos de teste

Bons estudos! Lembre-se de praticar a escrita dos métodos à mão também, pois na prova você não terá um computador para testar. Treine a lógica no papel.